Triggers

Exercises

01 – Exams and precedences

Consider the following relational schema:

EXAMINATION (<u>StudentID</u>, <u>Course</u>, Date, Grade)

PRECEDENCERULE (<u>PrecedingCourse</u>, <u>FollowingCourse</u>)

With the following integrity constraint:

it is forbidden to take an exam if there is a preceding course that has not been passed yet.

1. Find the operations that ***can*** violate the constraint.
2. Based on the common sense experience, choose the operation that will actually violate the constraint.
3. Write a trigger for that operation, that rolls back the effects when the constraint is violated.

It is forbidden to take an exam if there is a preceding course that has not been passed yet.

EXAMINATION (StudentID, Course, Date, Grade)

PRECEDENCERULE (PrecedingCourse, FollowingCourse)

EVENTS:

- Insert		EXAMINATION
- Update	on	PRECEDENCERULE
- Delete		

5 operations can possibly violate the integrity constraint:

- INSERT of a new EXAMINATION, when there are preceding courses that are not sustained.
- UPDATE of the Date or Course on EXAMINATION.
- DELETE of an EXAMINATION.
- UPDATE of an existing PRECEDENCERULE.
- INSERT of a new PRECEDENCERULE.

The operation that most typically will violate the rule is the first one.

Updates and deletions on examinations should occur exceptionally, and only to handle specific errors. Changes to precedence rules, instead, may affect older exams. One solution to this issue is to extend the schema with a ***date*** *from which the precedence rule is activated and a date in which the rule is to be considered invalidated.*

Event: INSERT on EXAMINATION

EXAMINATION (StudentID, Course, Date, Grade)

PRECEDENCERULE (PrecedingCourse, FollowingCourse)

```
create trigger MissingPrecedence
before insert on EXAMINATION
for each row
when exists
        ( select *
        from PRECEDENCERULE
        where FollowingCourse = new.Course AND
        PrecedingCourse not in (   select Course
                                   from EXAMINATION
                                   where StudentID= new.StudentID
                                   AND Date < new.Date ) )
rollback;
```

if there is a PrecedenceRule

and the current student did not register the preceding course

Rollback/raise app error

02 - Play with me

Consider the following schema describing a system for hiring rehearsal rooms to musical groups. Start and end time of reservations contain only hours (minutes are always “00”). Use of rooms can only happen if there is a corresponding reservation, but can start later or end earlier. All rooms open at 7:00 and close at 24:00.

User (<u>SSN</u>, Name, Email, Type)

Reservation (<u>UserSSN</u>, <u>RoomCode</u>, <u>Date</u>, <u>StartTime</u>, EndTime)

Room (<u>RoomId</u>, Type, CostPerHour)

1) Write a trigger that <u>*prevents the reservation*</u> of already booked rooms

2) Suppose that usage data are inserted into a table Usage only after the room has been **actually** used. Enrich the schema <u>to track the number of hours that have been reserved but not used</u> by each user and write a (set of) trigger(s) that track such a number and <u>set the “type” of a user to “unreliable”</u> when he totalizes 50 hours of unused reservations.

1) Write a trigger that *prevents the reservation* of already booked rooms

Reservation (UserSSN, RoomCode, Date, StartTime, EndTime)

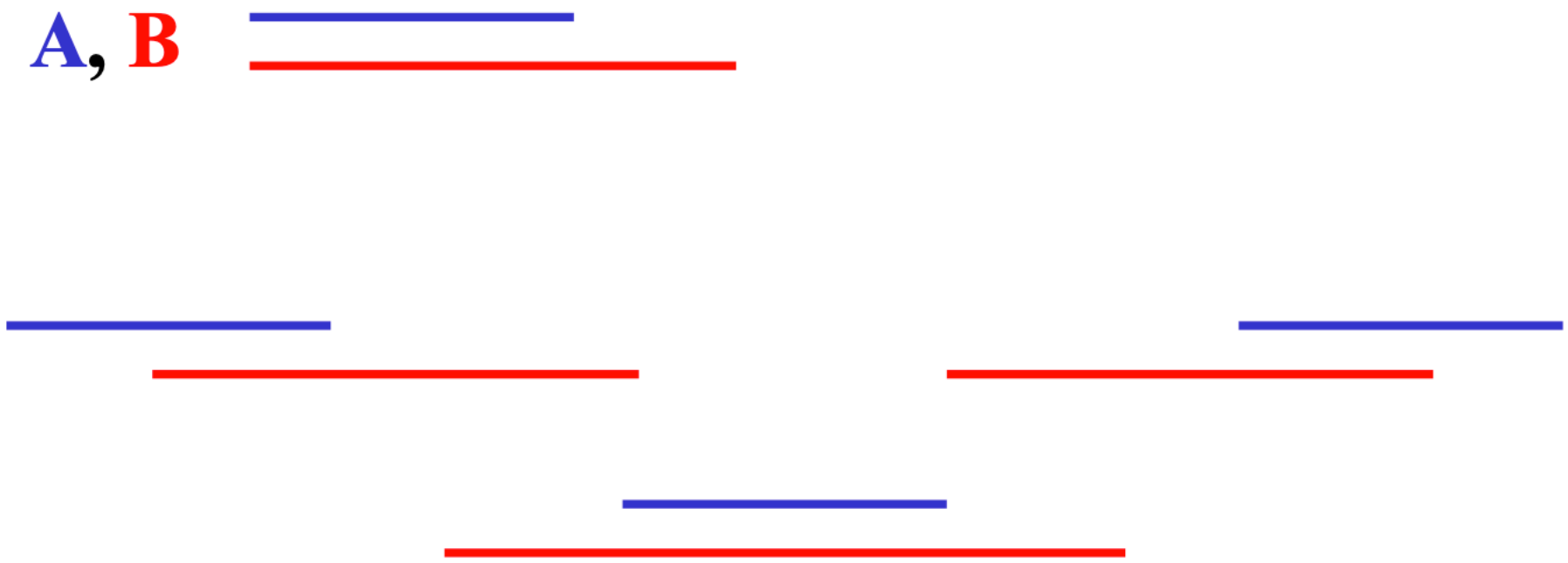
```
create trigger ThouShallNotBook
before insert on Reservation
for each row
when exists (   select *
                        from Reservation
                        where RoomCode = new.RoomCode
                        and Date = new.Date and
                        ??
                )
rollback;
```

We can use a "before" semantics to impose an early rollback of the transaction

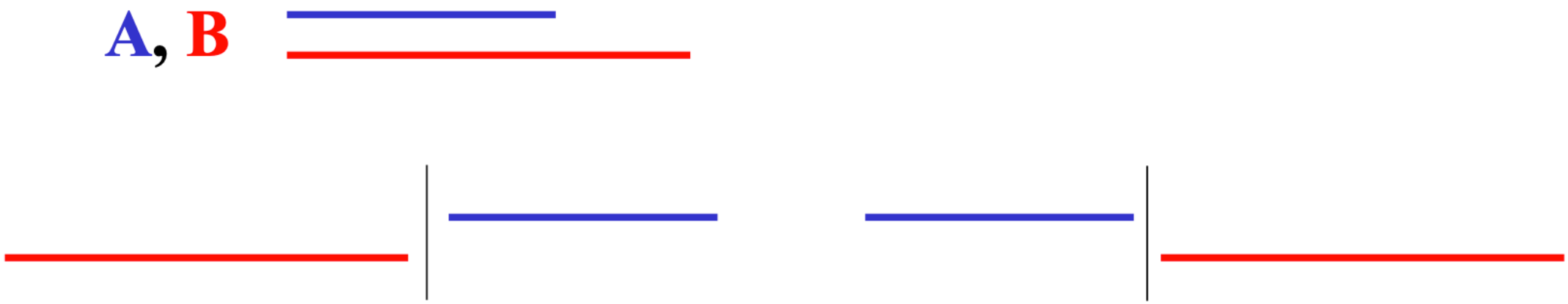
We need to capture reservations for the same room with overlapping periods

Two intervals overlap when..?

What about overlapping intervals?



...*not* overlapping intervals?



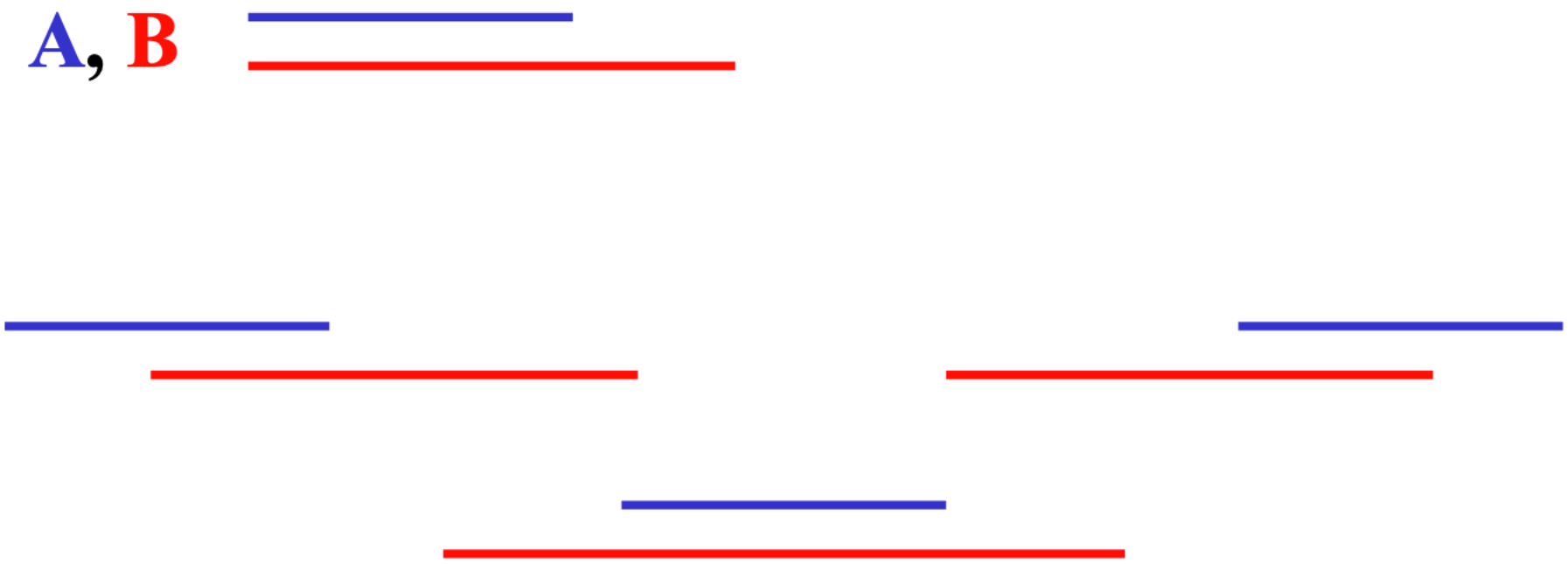
A.start > B.end or B.start > A.end

De Morgan

NOT OVERLAPPING: not(A.start <= B.end and B.start <= A.end)

OVERLAPPING: A.start <= B.end and B.start <= A.end)

What about overlapping intervals?



A.start <= B.end and B.start <= A.end)

1) Write a trigger that *prevents the reservation* of already booked rooms

Reservation (UserSSN, RoomCode, Date, StartTime, EndTime)

```
create trigger ThouShallNotBook
before insert on Reservation
for each row
when exists (   select *
                from Reservation
                where RoomCode = new.RoomCode
                and Date = new.Date and
                StartTime < new.EndTime and EndTime > new.StartTime
        )
rollback;
```

We can use a "before" semantics to impose an early rollback of the transaction

We need to capture reservations for the same room with overlapping periods

Two intervals overlap

2) Suppose that usage data are inserted into a table Usage only after the room has been **actually** used. Enrich the schema to track the number of hours that have been reserved but not used by each user and write a (set of) trigger(s) that track such a number and set the "type" of a user to "unreliable" when he totalizes 50 hours of unused reservations.

User (SSN, Name, Email, Type)
Reservation (UserSSN, RoomCode, Date, StartTime, EndTime)

Usage (UserSSN, RoomCode, Date, StartTime, EndTime)

Unused hours can be counted via queries, without further schema modifications. For efficiency reasons, however, we may want to calculate the number incrementally, e.g., by adding a "WastedHours" field to table User.

User (SSN, Name, Email, Type, WastedHours)

How do we match reservations and usages?

We assume that (1) the previous trigger guarantees reservations to be correct and not overlapping and that (2) actual usages of each room only and always happen within the limits of a reserved interval, and by those who actually reserved the room.

Reservation (<u>UserSSN, RoomCode, Date, StartTime</u>, EndTime)

User (<u>SSN</u>, Name, Email, Type, WastedHours)

Usage (<u>UserSSN, RoomCode, Date, StartTime</u>, EndTime)

1) Track the number of hours that have been reserved but not used by each user

NOT USED =

- Not used at all
- Partially used

2) set the "type" of a user to "unreliable" when he totalizes 50 hours of unused reservations

Reservation (UserSSN, RoomCode, Date, StartTime, EndTime)
User (SSN, Name, Email, Type, WastedHours)
Usage (UserSSN, RoomCode, Date, StartTime, EndTime)

Track the number of hours that have been reserved but not used by each user (PARTIALLY USED)
Hypothesis: Usage for NOT USED is inserted with EndTime = StartTime

```
create trigger UpdateWastedHours
after insert on Usage
for each row
  update User
  set WastedHours = WastedHours + ( select EndTime – StartTime – ( new.EndTime – new.StartTime )
                                    from Reservation
                                    where RoomCode=new.RoomCode and Date = new.Date
                                    and StartTime<= new.StartTime and EndTime >= new.EndTime )

  where SSN = new.UserSSN
```

*This is the simplest option: the field is **updated at each insertion**, possibly with a zero- increment when users are on time.*
*Capturing zero-increments may be done in the **when** clause*

Reservation (UserSSN, RoomCode, Date, StartTime, EndTime)
User (SSN, Name, Email, Type, WastedHours)
Usage (UserSSN, RoomCode, Date, StartTime, EndTime)

```
create trigger UpdateWastedHours
after insert on Usage
for each row
declare X integer;
when 0 < ( select EndTime – StartTime – ( new.EndTime – new.StartTime ) into X
                                        from Reservation
                                        where RoomCode=new.RoomCode and Date = new.Date
                                        and StartTime<= new.StartTime and EndTime >= new.EndTime )

  update User
  set WastedHours = WastedHours + X
  where SSN = new.UserSSN
```

Then, the only missing part is the monitoring of the threshold:

Reservation (UserSSN, RoomCode, Date, StartTime, EndTime)
User (SSN, Name, Email, Type, WastedHours)
Usage (UserSSN, RoomCode, Date, StartTime, EndTime)

```
create trigger UpdateType
after update of WastedHours on User
for each row
when old.WastedHours < 50 and new.WastedHours >= 50

update User
  set Type = "Unreliable"
  where SSN = old.SSN;
```

03 – MinDistance (June 21st, 2019)

PAGE (<u>PageID</u>, Content, HomeY/N, MinDistanceFromHome)

HYPERLINK (<u>FromPageID, ToPageID</u>)

The above relational represents the map of a website: it contains pages and hyperlinks between pages.

For each page, the **minimum distance from the home page** is computed (it is not inserted by the client application). The insertion/deletion of hyperlinks may possibly change such distances in the whole graph. Initially the database is empty. Design a set of triggers that correctly keep the distance from the home page automatically updated when:

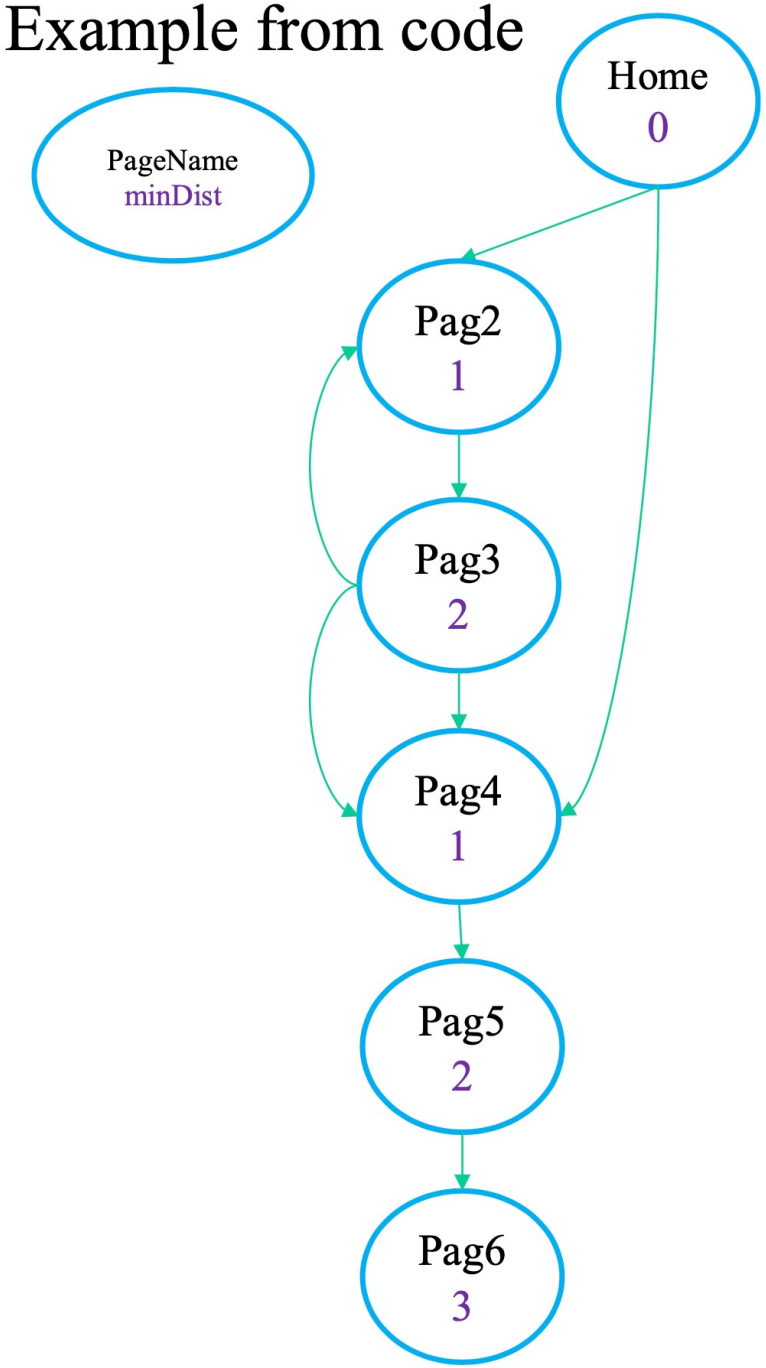
a) the home page is created;

b) a new hyperlink is added.

Notice that the map is a graph and therefore each page may have links towards several pages and incoming links from several pages. If needed, you may possibly enrich the schema with further attributes/tables.

Briefly discuss the termination of the trigger system.

Example from code




```
/*  ADD TWO PAGES, ONE AS HOME TO FIRE THE TRIGGER  1 (0) */
insert into db2_hyperlinks.page ("Content", "HomeYN") values ('Home', 'Y');
insert into db2_hyperlinks.page ("Content", "HomeYN") values ('pag2', 'N');
select * from db2_hyperlinks.page order by "PageID";

/*  ADD AN HYPERLINK 1 (0) --> 2 (1) */
insert into db2_hyperlinks.hyperlink values (1, 2);
select * from db2_hyperlinks.page order by "PageID";

/* ADD A NEW PAGE */
insert into db2_hyperlinks.page ("Content", "HomeYN") values ('pag3', 'N');
select * from db2_hyperlinks.page order by "PageID";

/*  ADD AN HYPERLINK 2 (1) --> 3 (2) */
insert into db2_hyperlinks.hyperlink values (2, 3);
select * from db2_hyperlinks.page order by "PageID";

/*  ADD A NEW PAGE AND
   ADD AN HYPERLINK 3 (2) --> 4 (3) */
insert into db2_hyperlinks.page ("Content", "HomeYN") values ('pag4', 'N');
insert into db2_hyperlinks.hyperlink values (3, 4);
select * from db2_hyperlinks.page order by "PageID";

/*  ADD A NEW PAGE AND
   ADD AN HYPERLINK 4 (3) --> 5 (4) */
insert into db2_hyperlinks.page ("Content", "HomeYN") values ('pag5', 'N');
insert into db2_hyperlinks.hyperlink values (4, 5);
select * from db2_hyperlinks.page order by "PageID";

/*  ADD A HYPERLINK  1 (0) --> 4 (prev: 3 curr: 1)
   TRIGGER SHOULD ALSO UPDATE PAGE 5 (prev: 4 curr: 2*/
insert into db2_hyperlinks.hyperlink values (1, 4);
select * from db2_hyperlinks.page order by "PageID";

/*  ADD A NEW PAGE AND
   ADD AN HYPERLINK 5 (2) --> 6 (3) */
insert into db2_hyperlinks.page ("Content", "HomeYN") values ('pag6', 'N');
insert into db2_hyperlinks.hyperlink values (5, 6);
select * from db2_hyperlinks.page order by "PageID";

/* ADD AN HYPERLINK WITH A CYCLE 3 (2) --> 2 (1), the cycle stops because 2+1 > 1 */
insert into db2_hyperlinks.hyperlink values (3, 2);
select * from db2_hyperlinks.page order by "PageID";

/* ADD AN HYPERLINK WITH A CYCLE 4 (1) --> 3 (2), the cycle stops because 1+1 == 2 */
insert into db2_hyperlinks.hyperlink values (4, 3);
select * from db2_hyperlinks.page order by "PageID";
```

PAGE (<u>PageID</u>, Content, HomeY/N, MinDistanceFromHome)
HYPERLINK (<u>FromPageID, ToPageID</u>)

a) the home page is created, need to set minDist = 0 (T1)

```
CREATE TRIGGER InsHomePage
BEFORE INSERT ON PAGE
FOR EACH ROW
WHEN (new.HomeY/N)="Y"

new.MinDistanceFromHome = 0;
```

we can assume that other triggers do the needed check on the home page e.g., uniqueness etc.

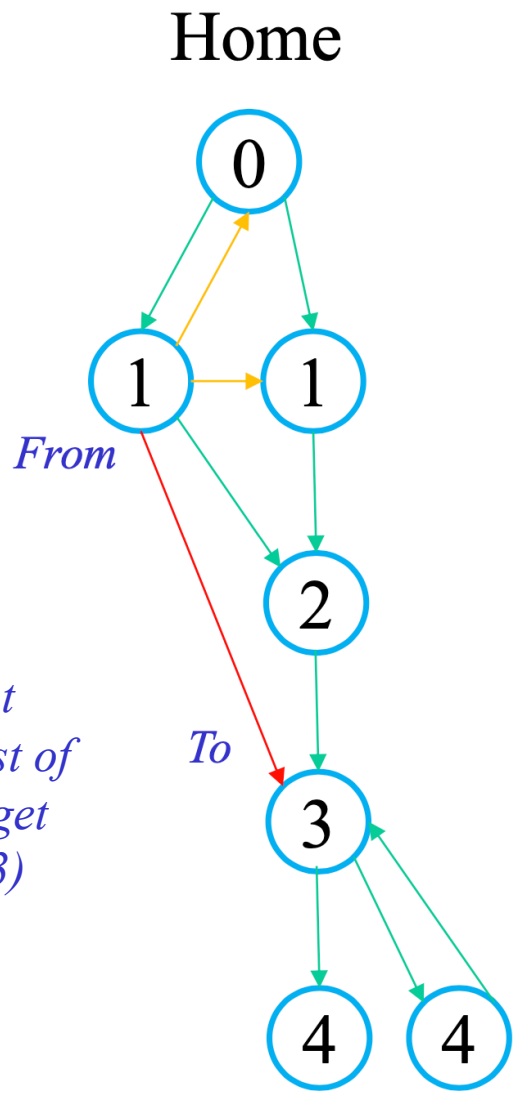
PAGE (PageID, Content, HomeY/N, MinDistanceFromHome)
HYPERLINK (FromPageID, ToPageID)

b) a new hyperlink is added (T2)

```
CREATE TRIGGER InsHyperlink
AFTER INSERT ON HYPERLINK
FOR EACH ROW
WHEN (SELECT MinDistanceFromHome
            FROM PAGE
            WHERE PageID=new.FromPageID) + 1) <
            (SELECT MinDistanceFromHome FROM PAGE
            WHERE PageID=new.ToPageID)
UPDATE PAGE
SET MinDistanceFromHome = 1+
            (SELECT MinDistanceFromHome FROM PAGE
            WHERE PageID=new.FromPageID)
WHERE PageID=new.ToPageID;
```

Current MinDist of the source node (1) + 1 = 2

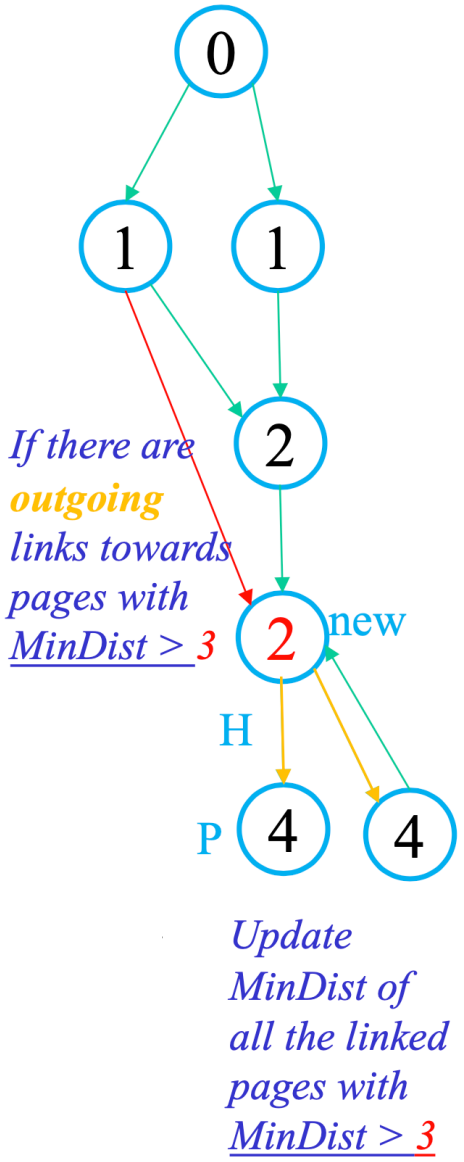
Current MinDist of the target node (3)



PAGE (PageID, Content, HomeY/N, MinDistanceFromHome)
HYPERLINK (FromPageID, ToPageID)

b) Recursive updates (T3)

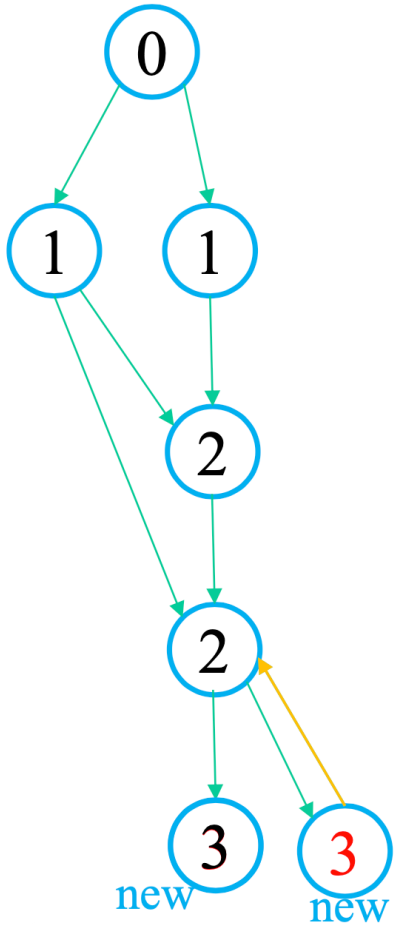
```
CREATE TRIGGER UpdateDistance
AFTER UPDATE OF MinDistanceFromHome ON PAGE
FOR EACH ROW
WHEN EXISTS (SELECT *
                FROM PAGE AS P JOIN HYPERLINK AS H ON
                H.ToPageID=P.PageID
                WHERE H.FromPageID=new.PageID AND
                P.MinDistanceFromHome > new.MinDistanceFromHome + 1)
UPDATE PAGE
SET MinDistanceFromHome = new.MinDistanceFromHome + 1
        WHERE PageID IN (SELECT P.PageID
        FROM PAGE AS P JOIN HYPERLINK AS H ON
        H.ToPageID=P.PageID
        WHERE H.FromPageID=new.PageID AND
        P.MinDistanceFromHome > new.MinDistanceFromHome + 1);
```



PAGE (PageID, Content, HomeY/N, MinDistanceFromHome)
HYPERLINK (FromPageID, ToPageID)

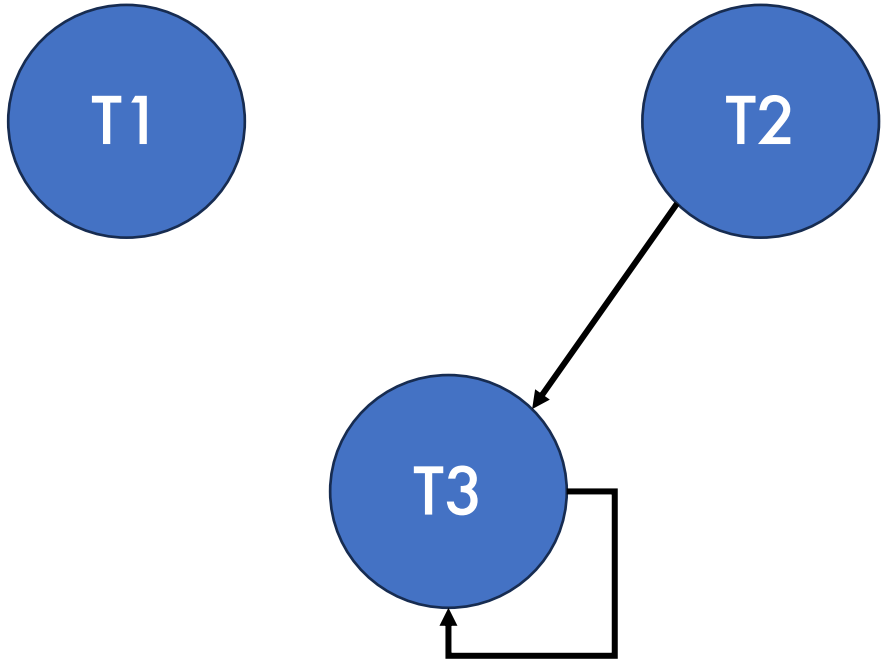
b) Recursive updates (T3)

```
CREATE TRIGGER UpdateDistance
AFTER UPDATE OF MinDistanceFromHome ON PAGE
FOR EACH ROW
WHEN EXISTS (SELECT *
                FROM PAGE AS P JOIN HYPERLINK AS H ON
                H.ToPageID=P.PageID
                WHERE H.FromPageID=new.PageID AND
                P.MinDistanceFromHome > new.MinDistanceFromHome + 1)
UPDATE PAGE
SET MinDistanceFromHome = new.MinDistanceFromHome + 1
        WHERE PageID IN (SELECT P.PageID
        FROM PAGE AS P JOIN HYPERLINK AS H ON
        H.ToPageID=P.PageID
        WHERE H.FromPageID=new.PageID AND
        P.MinDistanceFromHome > new.MinDistanceFromHome + 1);
```



*If there are **outgoing** links towards pages with MinDist > 4*

Termination of the trigger system



T3 is the only trigger that could be non-terminating;

Nevertheless, the WHEN conditions in T3 avoid updating cycles and T3 will eventually terminate reaching the leaves.

04 – Grade Registration (February 19th, 2019)

STUDENT (StudentID, LastName, Name, Country)

COURSE (CourseID, Name, Professor)

EXAM (CourseID, ExamDate, ClassRoom, Time, PublicationDate, ClosedY/N)

GRADES (CourseID, StudentID, ExamDate, Grade, RejectedY/N)

STATSBYDATE (ExamDate, TotalNumberOfStudents)

STATSBYEXAM (CourseID, ExamDate, AveragePublishedGrade, AverageFinalGrade)

The database above supports (a simplified version of) the exam registration process in a technical university. Professors insert the grades of each exam through a Web form: they fill in the data of ***all*** the students who passed the exam on that date, and then click on the SEND button: all data are sent to the server and inserted into table GRADES with a "bulk" command like:

```
INSERT INTO GRADES (courseID,StudentID,Date,Grade)
VALUES (808080,12345,19/02/2019,18), (808080,12543,19/02/2019,30),...;
```

and the PublicationDate of the exam is set. Students can reject their grade within 5 days, and grades can possibly be updated by professors in this period. After five days from publication the exam is closed.

04 – Grade Registration (February 19th, 2019)

STUDENT (StudentID, LastName, Name, Country)

COURSE (CourseID, Name, Professor)

EXAM (CourseID, ExamDate, ClassRoom, Time, PublicationDate, ClosedY/N)

GRADES (CourseID, StudentID, ExamDate, Grade, RejectedY/N)

STATSBYDATE (ExamDate, TotalNumberOfStudents)

STATSBYEXAM (CourseID, ExamDate, AveragePublishedGrade, AverageFinalGrade)

Write a set of triggers that allow keeping the following statistics updated:

a) for each date, the total number of students who did the exam (regardless of the acceptance/rejection of the grade);

b) for each exam, the average grades initially published by the professor and the average final grade of the exam (considering only the accepted grades) when it is "closed".

Only the last 10 statistics for each course are kept in table STATSBYEXAM.

STUDENT (StudentID, LastName, Name, Country)

COURSE (CourseID, Name, Professor)

EXAM (CourseID, ExamDate, ClassRoom, Time, PublicationDate, ClosedY/N)

GRADES (CourseID, StudentID, ExamDate, Grade, RejectedY/N)

STATSBYDATE (ExamDate, TotalNumberOfStudents)

STATSBYEXAM (CourseID, ExamDate, AveragePublishedGrade, AverageFinalGrade)

a) Write a set of triggers that allow keeping the following statistics updated: a) for each date, the total number of students who did an exam (regardless of the acceptance/rejection of the grade);

```
CREATE TRIGGER StatisticsByDate                    // for each row version
AFTER INSERT on GRADES
FOR EACH ROW
BEGIN
        IF (EXISTS SELECT * FROM STATSBYDATE
                WHERE ExamDate = new.ExamDate)
           UPDATE STATSBYDATE
                SET TotalNumberOfStudenst = TotalNumberOfStudents + 1
                WHERE ExamDate = new.ExamDate;
        ELSE
           INSERT INTO STATSBYDATE VALUES (new.ExamDate, 1);
END;
```

If it is the first exam of the date, a new tuple is inserted, otherwise the tuple for that date is updated.

If the same student participates to more than one exam on the same date, s/he will be counted once for each exam.

STUDENT (StudentID, LastName, Name, Country)

COURSE (CourseID, Name, Professor)

EXAM (CourseID, ExamDate, ClassRoom, Time, PublicationDate, ClosedY/N)

GRADES (CourseID, StudentID, ExamDate, Grade, RejectedY/N)

STATSBYDATE (ExamDate, TotalNumberOfStudents)

STATSBYEXAM (CourseID, ExamDate, AveragePublishedGrade, AverageFinalGrade)

```
CREATE TRIGGER StatisticsByDate                    // for each statement version
AFTER INSERT on GRADES
REFERENCING NEW TABLE as INSERTEDGRADES
FOR EACH STATEMENT

BEGIN
        declare newDate date;
        SELECT DISTINCT ExamDate into newDate from INSERTEDGRADES;
IF (NOT EXISTS SELECT * FROM STATSBYDATE WHERE ExamDate = newDate)
          INSERT INTO STATSBYDATE VALUES (newDate, select count(*) from INSERTEDGRADES)
ELSE
        UPDATE STATSBYDATE
        SET TotalNumberOfStudenst = TotalNumberOfStudents + (select count(*) from INSERTEDGRADES)
        WHERE ExamDate = newDate;
END;
```

newDate = ???
Assumption: with each insert command the date of the inserted exams is the same. It is saved in a variable

STUDENT (StudentID, LastName, Name, Country)
COURSE (CourseID, Name, Professor)
EXAM (CourseID, ExamDate, ClassRoom, Time, PublicationDate, ClosedY/N)
GRADES (CourseID, StudentID, ExamDate, Grade, RejectedY/N)
STATSBYDATE (ExamDate, TotalNumberOfStudents)
STATSBYEXAM (CourseID, ExamDate, AveragePublishedGrade, AverageFinalGrade)

b) for each exam, the **average grades initially published** by the professor and the **average final grade** of the exam (considering only the accepted grades) when it is "closed".

```
CREATE TRIGGER AveragePublishedExams
FOR EACH STATEMENT
AFTER INSERT on GRADES
REFERENCING NEW TABLE as INSERTEDGRADES
INSERT INTO STATSBYEXAM
        SELECT CourseID, ExamDate, Average(Grade), 0.0
        FROM INSERTEDGRADES
        GROUP BY CourseID, ExamDate;
```

EXAM (CourseID, ExamDate, ClassRoom, Time, PublicationDate, ClosedY/N)

GRADES (CourseID, StudentID, ExamDate, Grade, RejectedY/N)

STATSBYEXAM (CourseID, ExamDate, AveragePublishedGrade, AverageFinalGrade)

After publication grades may change. However, we need to compute the new average grade as soon as the publication is closed.

```
CREATE TRIGGER FinalAverage
AFTER UPDATE of CLOSED on EXAM
FOR EACH ROW
WHEN old.Closed="N" and new.Closed="Y"

UPDATE STATSBYEXAM

        SET AverageFinalGrade = (SELECT Average(grade)
                                 FROM GRADES
                                 WHERE CourseID=new.CourseID AND
                                 ExamDate=New.ExamDate AND
                                 Rejected="N")

        WHERE CourseID=new.CourseID AND ExamDate=New.ExamDate;
```

STATSBYEXAM (<u>CourseID, ExamDate</u>, AveragePublishedGrade, AverageFinalGrade)

To maintain only the last 10 statistics of each course, we can use the event "insert on StatsByExam" and delete the oldest exam of the same course.

```
CREATE TRIGGER Last10
AFTER INSERT on STATSBYEXAM
FOR EACH ROW
WHEN (10 < SELECT COUNT(*)
                FROM STATSBYEXAM
                WHERE CourseID=new.CourseID)
DELETE FROM STATSBYEXAM
        WHERE CourseID=new.CourseID AND
        ExamDate=(SELECT MIN(ExamDate)
                FROM STATSBYEXAM
                WHERE CourseID=new.CourseID);
```

05 – University applications (July 7th, 2020)

Consider the following schema, where primary keys are identified by capital letters. Students Application to university programs. Programs accept a maximum number of students according to a first- come, first-served strategy.

Program(**PROG_ID**, Name, MaxNumberOfStudents)

Application(**STUD_ID**, **Prog_ID**, Accepted) // Accepted can be 'Y' or 'N'

WaitingList(**STUD_ID**, **Prog_ID**, Timestamp)

Write a set of triggers to manage these two particular cases:

a) A new student applies to a program, but the program has already accepted the maximum number of students: <u>new students cannot be accepted</u>, so they are inserted in the waiting list;

b) A program decides to increase the maximum number of students: students in the waiting list for that program are accepted, based on the timestamp of their application.

a) the program has already accepted the maximum number of students: new students cannot be accepted, so they are inserted in the waiting list

Program(**PROG_ID**, Name, MaxNumberOfStudents)
Application(**STUD_ID, Prog_ID**, Accepted)
WaitingList(**STUD_ID, Prog_ID**, Timestamp)

Need AFTER INSERT because BEFORE triggers cannot modify directly the db

```
CREATE TRIGGER StudNotAccepted
AFTER INSERT ON Application
FOR EACH ROW
WHEN
        (SELECT count(*)
        FROM Application
        WHERE Accepted="Y" and Prog_ID = NEW.Prog_ID)
        >=
        (SELECT MaxNumberOfStudents
        FROM Program
        WHERE Prog_ID=NEW.Prog_ID)
BEGIN
        UPDATE Application SET Accepted="N"
                WHERE STUD_ID=NEW.STUD_ID and Prog_ID = new.Prog_ID;
        INSERT INTO WaitingList VALUES
                (NEW.STUD_ID, NEW.Prog_ID, SysDateTime())
END;
```

If current number of applications >= Maximum number of students

Cannot be accepted

Waiting list

b) A program decides to increase the maximum number of students: students in the waiting list for that program are accepted, based on the timestamp of their application.

Program(**PROG_ID**, Name, MaxNumberOfStudents)
Application(**STUD_ID, Prog_ID**, Accepted)
WaitingList(**STUD_ID, Prog_ID**, Timestamp)

```
CREATE TRIGGER AcceptFromWaitingList
AFTER UPDATE OF MaxNumberOfStudents ON Program
FOR EACH ROW
WHEN EXISTS (SELECT * FROM WaitingList where Prog_ID = new.Prog_ID) AND
new.MaxNumberOfStudents > old.MaxNumberOfStudents
BEGIN
UPDATE Application SET Accepted = 'Y' WHERE STUD_ID IN
        (SELECT STUD_ID FROM WaitingList where Prog_id = new.PROG_ID
        ORDER BY Timestamp
        LIMIT (new.MaxNumberOfStudents - old.MaxNumberOfStudents))
DELETE FROM WaitingList WHERE STUD_ID IN
        (SELECT STUD_ID FROM WaitingList where Prog_id = new.PROG_ID ORDER BY Timestamp
        LIMIT (new.MaxNumberOfStudents – old.MaxNumberOfStudents))
END
```

If there are students in the waiting list

Update first N students in the waiting list

Delete first N students from the waiting list